HW3_Problem1

January 28, 2026

1 PROBLEM 1

This problem concentrates on training models for the MNIST dataset using Decision Trees, Ensemble Models and Dimensionality reduction: Please fix the random state to 42 where required.

1.1 1.1

1.1.1 Open a Jupyter-notebook load the the MNIST dataset and split it to 80% training and 20% test parts using stratisfied splitting with a fixed random state. Retain only the 10000 first entries of the resulting training set and the first 2000 from the test set. Convert labels from array of strings to array of 64-bit integers.

[1]:
```
## Collection of imports for Problem 1
# import numpy as np
# import pandas as pd
# import matplotlib.pyplot as plt
# import time
# from sklearn.datasets import fetch_openml
# from sklearn.model_selection import train_test_split, GridSearchCV
# from sklearn.metrics import accuracy_score, f1_score
# from sklearn.tree import DecisionTreeClassifier
# from sklearn.ensemble import GradientBoostingClassifier
# from sklearn.cluster import KMeans
# from sklearn.pipeline import make_pipeline, Pipeline
# from sklearn.preprocessing import StandardScaler
# from sklearn.decomposition import PCA
```

Load MNIST dataset

[2]:
```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

[3]:
```
X_full, y_full = mnist["data"], mnist["target"]
```

[4]:
```
import numpy as np

y_full = y_full.astype(np.int64) # Convert labels to array of 64-bit integers
```

Split dataset into 80% training and 20% test while stratifying on labels. Stratification ensures each label is proportionally represented in train and test sets.

[5]:
```
from sklearn.model_selection import train_test_split

X_train_full, X_test_full, y_train_full, y_test_full = train_test_split(X_full,␣
↪y_full, test_size=0.2, stratify=y_full, random_state=42)
```

Retain only 10,000 training and 2,000 test samples

[6]:
```
X_train, y_train = X_train_full[:10000], y_train_full[:10000]
X_test, y_test = X_test_full[:2000], y_test_full[:2000]

print(f"Training set shape: {X_train.shape}, Labels shape: {y_train.shape}")
print(f"Test set shape: {X_test.shape}, Labels shape: {y_test.shape}")
print(f"Label dtype: {y_train.dtype}")
```

```
Training set shape: (10000, 784), Labels shape: (10000,)
Test set shape: (2000, 784), Labels shape: (2000,)
Label dtype: int64
```

1.2 1.2

1.2.1 Perform a Grid Search with 5-fold cross validation, maximum features taking the values [100, 150, 200], and the maximum depth the values [2, 4, 5], for a Decision Tree Classifier, using the Entropy criterion and fixed random state. Print the accuracy score, the F1 - score (with average parameter set to "macro"), with respect to test data, and the values of the best parameters for the best estimator. Print all scores for all combinations and discuss the results.

Set the Grid Search parameters

[7]:
```
gs_parameters = {'max_features' : [100, 150, 200], 'max_depth' : [2, 4, 5]}
```

Perform Grid Search with 5-fold cross-validation

[8]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, f1_score

clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

grid_search = GridSearchCV(estimator=clf,
                           param_grid=gs_parameters,
                           cv=5,
                           scoring='accuracy',
                           n_jobs=-1) # use n_jobs=-1 to use all CPU cores

grid_search.fit(X_train, y_train)
```

[8]:
```
GridSearchCV(cv=5,
             estimator=DecisionTreeClassifier(criterion='entropy',
                                              random_state=42),
             n_jobs=-1,
             param_grid={'max_depth': [2, 4, 5],
                         'max_features': [100, 150, 200]},
             scoring='accuracy')
```

Retrieve the best estimator from the Grid Search and use it to make predictions on the test data

[9]:
```
# Best estimator
best_clf = grid_search.best_estimator_
# Predictions on test data
y_pred = best_clf.predict(X_test)
```

Fit the best estimator from the Grid Search, to time it for question 1.6.

[10]:
```
import time
start_time=time.time()
best_clf.fit(X_train, y_train)
end_time=time.time()
dt_training_time=end_time - start_time
```

Evaluate accuracy and F1-score on the test set

[11]:
```
accuracy = accuracy_score(y_test, y_pred)
f1_macro = f1_score(y_test, y_pred, average='macro')

print(f"Decision Tree Best Parameters: {grid_search.best_params_}")
print(f"Decision Tree Accuracy on test set: {accuracy:.4f}")
print(f"Decision Tree F1 Score (macro) on test set: {f1_macro:.4f}")
print(f"Decision Tree Training Time (s): {dt_training_time:.4f}")
```

```
Decision Tree Best Parameters: {'max_depth': 5, 'max_features': 150}
Decision Tree Accuracy on test set: 0.6760
Decision Tree F1 Score (macro) on test set: 0.6639
Decision Tree Training Time (s): 0.1830
```

Print all parameter combinations and their scores

[12]:
```
import pandas as pd

pd.DataFrame(grid_search.cv_results_).head(5)
```

[12]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	0.211800	0.009950	0.015399	0.003262	
1	0.236800	0.016972	0.014799	0.003488	
2	0.264399	0.010670	0.015002	0.003521	
3	0.289401	0.021010	0.021198	0.006968	
4	0.345800	0.016655	0.011599	0.001743	

```
   param_max_depth  param_max_features                                  params  \
0                2                 100  {'max_depth': 2, 'max_features': 100}
1                2                 150  {'max_depth': 2, 'max_features': 150}
2                2                 200  {'max_depth': 2, 'max_features': 200}
3                4                 100  {'max_depth': 4, 'max_features': 100}
4                4                 150  {'max_depth': 4, 'max_features': 150}

   split0_test_score  split1_test_score  split2_test_score  split3_test_score  \
0             0.3365             0.3050             0.3140              0.324
1             0.3090             0.3075             0.3245              0.314
2             0.3195             0.3220             0.3265              0.321
3             0.5720             0.5625             0.5945              0.579
4             0.5785             0.5735             0.5940              0.594

   split4_test_score  mean_test_score  std_test_score  rank_test_score
0             0.3015           0.3162        0.012801                9
1             0.3260           0.3162        0.007711                8
2             0.3260           0.3230        0.002775                7
3             0.5050           0.5626        0.030636                6
4             0.5475           0.5775        0.017097                5
```

[13]:
```
results = pd.DataFrame(grid_search.cv_results_)
results[['param_max_features', 'param_max_depth', 'mean_test_score']]
```

[13]:
```
   param_max_features  param_max_depth  mean_test_score
0                 100                2           0.3162
1                 150                2           0.3162
2                 200                2           0.3230
3                 100                4           0.5626
4                 150                4           0.5775
5                 200                4           0.5841
6                 100                5           0.6506
7                 150                5           0.6605
8                 200                5           0.6546
```

We observe that a shallow tree depth of max_depth=2 results in underfitting, providing significantly lower test scores (around 32%). Increasing the depth to 4 leads to a notable improvement in performance, and further increasing it to 5 results in the best scores, confirming that deeper trees help the model learn more meaningful patterns. Regarding max_features, the relationship is not strictly linear. While higher values (200) tend to achieve the best results when max_depth=5 (65.46%), the improvement across different values is not consistent across all depths. Generally, lower max_features can help reduce variance, but too low values may increase bias. Given these results, decision trees alone provide reasonable performance but are limited. To further enhance generalization and performance, we should explore ensemble methods, which can mitigate overfitting while leveraging the benefits of multiple trees.

1.3 1.3

1.3.1 Create a pipeline performing PCA with fixed random state, retaining 90% of the variance of the initial features included in training dataset, to reduce dimensionality of the dataset and train a Decision Tree with the depth parameter discovered in the previous question. Compute accuracy and F1 - score with respect to test data and compare with the previous results. Please discuss the advantages or disadvantages of using PCA.

[14]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline

scaler = StandardScaler()
pca = PCA(n_components=0.90, random_state=42)
dt_pca = DecisionTreeClassifier(criterion='entropy', max_depth=5,␣
 ↪random_state=42)

pca_pipeline = make_pipeline(scaler, pca, dt_pca)

pca_pipeline.fit(X_train, y_train)
```

[14]:
```
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('pca', PCA(n_components=0.9, random_state=42)),
                ('decisiontreeclassifier',
                 DecisionTreeClassifier(criterion='entropy', max_depth=5,
                                        random_state=42))])
```

Calculate the time need to fit the Decision Tree after scaling the data and performing PCA.

[15]:
```
X_train_scaled = scaler.fit_transform(X_train)

X_train_pca_time = pca.fit_transform(X_train_scaled)

start_time = time.time()
best_clf.fit(X_train_pca_time, y_train)
pca_dt_training_time = time.time() - start_time
```

Make predictions on the test set

[16]:
```
y_pred_pca = pca_pipeline.predict(X_test)
```

Evaluate performance on the test set

[17]:
```
from sklearn.metrics import accuracy_score, f1_score

accuracy_pca = accuracy_score(y_test, y_pred_pca)
f1_macro_pca = f1_score(y_test, y_pred_pca, average='macro')

print(f"Decision Tree Accuracy after PCA : {accuracy_pca:.4f}")
```

```
print(f"Decision Tree F1 Score (macro) after PCA : {f1_macro_pca:.4f}")
print(f"Decision Tree Training Time (s) after PCA: {pca_dt_training_time:.4f}")
```

```
Decision Tree Accuracy after PCA : 0.6720
Decision Tree F1 Score (macro) after PCA : 0.6682
Decision Tree Training Time (s) after PCA: 2.2240
```

Using PCA before training a Decision Tree in Problem 1.3 resulted in a minor accuracy drop (0.676 -> 0.672) but a slight F1-score improvement (0.6639 -> 0.6682), suggesting better class balance. However, training time increased significantly (0.187s -> 2.197s) due to the additional preprocessing step. While PCA is beneficial for reducing dimensionality, removing noise, and improving efficiency in high-dimensional datasets, it is particularly useful for linear models that rely on decorrelated features. Additionally, it can help mitigate overfitting in cases where redundant or highly correlated features are present.

Despite these benefits, PCA can discard valuable information by transforming data into new components, which may explain the slight accuracy decline. Decision Trees naturally select important features and handle high-dimensional data well, making PCA redundant in most cases. Here, the added computational cost did not translate into meaningful performance gains, highlighting that PCA is unnecessary for Decision Trees unless dealing with extreme feature spaces (e.g. 10,000+ features)

1.4 1.4

1.4.1 Perform PCA with the number of components dictated in the previous question, compress the training data and train a Gradient Boosting Classifier (GBC). The model should have a maximum depth of 2, 6 estimators and a learning rate equal to 1.0. The random state should be fixed where required. Discuss the results with respect to previously used Tree Classifier. How is it possible for GBC with shallow trees to outperform a deeper Tree Classifier?

Create a pipeline which standardizes the features to ensure all features contribute equally, applies Principal Component Analysis to reduce dimensionality, retaining 90% of the variance, and trains a Gradient Boosting Classifier, with shallow trees.

[18]:
```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, f1_score


gbc_pipeline = Pipeline([
    ('scaler', StandardScaler()),  # Standardize features before PCA to ensure␣
 ↪all features have equal importance
    ('pca', PCA(n_components=0.90, random_state=42)),  # Retain 90% variance
    ('gbc', GradientBoostingClassifier(
        max_depth=2,  # Use shallow trees, which helps reduce overfitting
        n_estimators=6,
```

```
        learning_rate=1.0,
        random_state=42
    ))
])


gbc_pipeline.fit(X_train, y_train)
```

[18]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('pca', PCA(n_components=0.9, random_state=42)),
                ('gbc',
                 GradientBoostingClassifier(learning_rate=1.0, max_depth=2,
                                            n_estimators=6, random_state=42))])
```

Calculate the time need to fit the Gradient Boosting Classifier after scaling the data and performing PCA.

[19]:
```
gbc = GradientBoostingClassifier(max_depth=2, n_estimators=6, learning_rate=1.
↪0, random_state=42)
start_time=time.time()
gbc.fit(X_train_pca_time, y_train)
end_time=time.time()
gbc_training_time = end_time - start_time
```

Make predictions on the test set

[20]:
```
y_pred_gbc = gbc_pipeline.predict(X_test)
```

Evaluate performance on the test set

[21]:
```
accuracy_gbc = accuracy_score(y_test, y_pred_gbc)
f1_macro_gbc = f1_score(y_test, y_pred_gbc, average='macro')

print(f"Accuracy with GBC: {accuracy_gbc:.4f}")
print(f"F1 Score (macro) with GBC: {f1_macro_gbc:.4f}")
print(f"Training Time (s): {gbc_training_time:.4f}")
```

```
Accuracy with GBC: 0.7940
F1 Score (macro) with GBC: 0.7903
Training Time (s): 23.4128
```

By combining PCA with a Gradient Boosting ensemble of shallow trees, we achieve the best results so far (~79%). Boosting iteratively refines weak learners, explaining the significant performance gain. This highlights the value of ensemble methods.

1.5 1.5

1.5.1 Reconstruct the first five images (digits) by using the output of PCA, of the previous question, and plot them along with their corresponding originals in the same figure and discuss. Perform KMeans Clustering, with 20 clusters, using the PCA transformed training data and plot the most representative digits in a Figure with 2 rows and 10 columns. The most representative digit of each cluster is the one that is closer to its corresponding centroid.

Extract the PCA object from the pipeline

[22]:
```
gbc_pipeline.named_steps['pca']
```

[22]:
```
PCA(n_components=0.9, random_state=42)
```

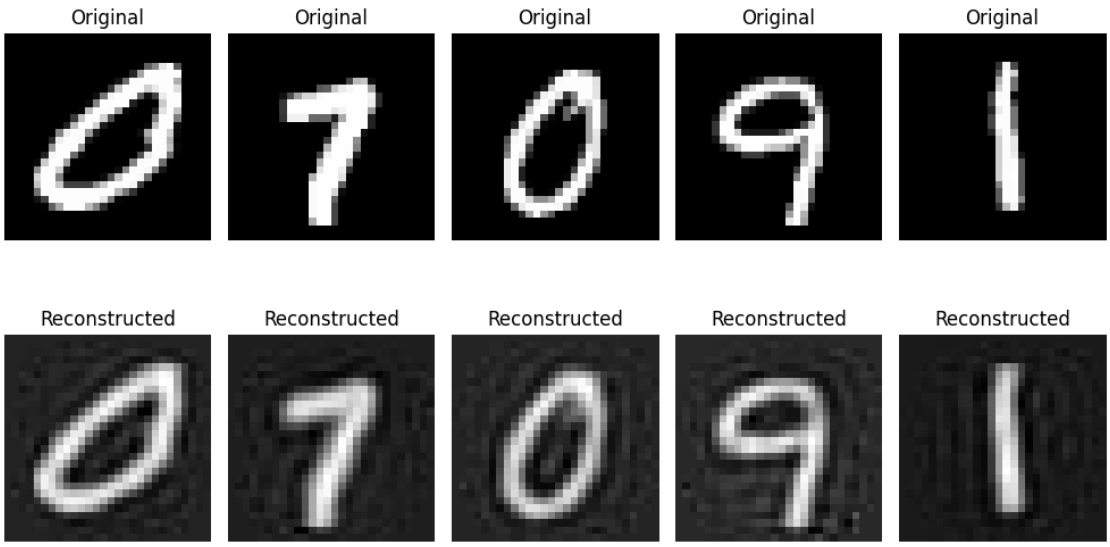
[23]:
```
pca = gbc_pipeline.named_steps['pca']
```

Transform and inverse transform the first 5 images

[24]:
```
X_train_pca = pca.transform(X_train[:5])  #Compress: Reduce the dimensionality␣
 ↪while retaining 90% of the variance
X_reconstructed = pca.inverse_transform(X_train_pca)  #Reconstruct: Reverse the␣
 ↪PCA transformation. We expect to get lower quality images on pixel level.
```

Plot original and reconstructed images

[25]:
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 5, figsize=(10, 6)) # Create a 2x5 grid of subplots
for i in range(5):
    # First row will be the Original images
    axes[0, i].imshow(X_train[i].reshape(28, 28), cmap='gray')  # display the␣
 ↪image from X_train, reshaping to 28x28
    axes[0, i].set_title("Original")
    axes[0, i].axis('off') # remove axis information for clearer visualization
    # Second row will be the Reconstructed images
    axes[1, i].imshow(X_reconstructed[i].reshape(28, 28), cmap='gray')
    axes[1, i].set_title("Reconstructed")
    axes[1, i].axis('off')

plt.tight_layout()
plt.show()
```



The reconstructed digits are visibly blurrier, but their overall form is intact. PCA at 90% variance discards some fine details while preserving the main shapes. This confirms PCA's ability to compress data while retaining the digit's essential structure

Retrieve scaler and PCA from the pipeline and apply them to the data

[26]:
```
scaler = gbc_pipeline.named_steps['scaler']
pca = gbc_pipeline.named_steps['pca']

X_train_scaled = scaler.transform(X_train)
X_train_pca = pca.transform(X_train_scaled)
```

Fit KMeans on the transformed data

Train KMeans with 20 clusters

[27]:
```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=20, random_state=42)
start_time=time.time()
kmeans.fit(X_train_pca)
```

[27]:
```
KMeans(n_clusters=20, random_state=42)
```

[28]:
```
end_time=time.time()
kmeans_training_time=end_time-start_time
```

Retrieve the clusters' centers

[29]:
```
centroids = kmeans.cluster_centers_
print(f"Centroids shape:",centroids.shape)
```

```
print(f"Centroids:\n",centroids)
```

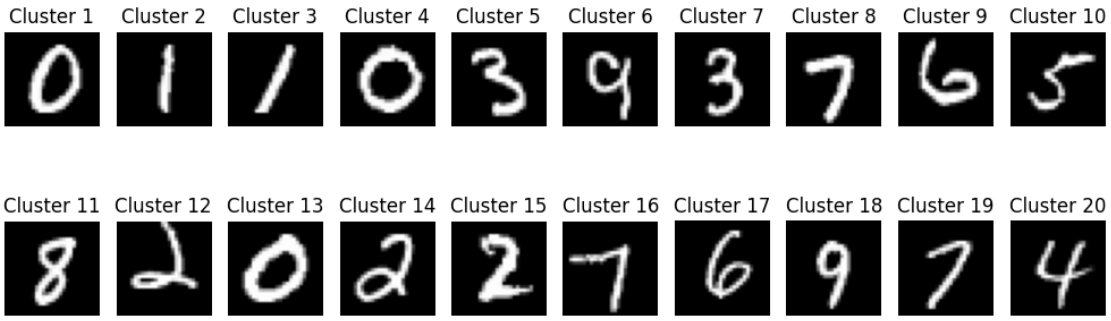
```
Centroids shape: (20, 202)
Centroids:
 [[ 7.76025358e+00 -3.08492192e+00 -1.31698545e+00 … -2.80128097e-02
   1.85147229e-02 -1.68453622e-02]
 [-6.79232309e+00 -8.60008687e-01  2.55982378e+00 …  9.65279677e-03
  -1.86900877e-02 -1.71743474e-02]
 [-6.17234799e+00 -3.26791005e+00  4.28607490e+00 … -3.70340451e-03
  -1.52001662e-02  2.51446045e-02]
 …
 [-4.70315542e+00  2.50640618e+00 -3.41116835e+00 …  7.23727166e-02
  -9.72970314e-03 -8.84886856e-03]
 [-3.85704467e+00 -1.55096573e+00 -6.19069117e+00 … -8.30950751e-02
   6.30512522e-02 -1.36439826e-02]
 [ 1.39201774e-01 -4.31901452e-01 -4.55670331e+00 …  1.44115973e-02
   8.06821835e-02  6.37723206e-03]]
```

Find the most representative image per cluster

[30]:
```
closest_samples = [] # Initialize an empty list to store the distances

for center in centroids: # Iterate over each centroid in the list of centroids
    distances = np.linalg.norm(X_train_pca - center, axis=1) # calculate the␣
 ↪Euclidean distance of every point in X_train_pca from the center
    closest_samples.append(np.argmin(distances)) # append the index of the␣
 ↪point in X_train_pca, which is closest to the center, to closest_samples list
```

Plot the most representative images

[31]:
```
fig, axes = plt.subplots(2, 10, figsize=(10, 4))
for i, idx in enumerate(closest_samples): # enumerate captures both the index i␣
 ↪of the loop, and the index idx of the element from closest_samples
    row, col = divmod(i, 10) # divomod will devide the index i of the loop by␣
 ↪10, returning the quotient to the row, and the remainder to the column
    axes[row, col].imshow(X_train[idx].reshape(28, 28), cmap='gray') # display␣
 ↪the image from X_train, reshaping to 28x28
    axes[row, col].axis('off') # remove axis information for clearer␣
 ↪visualization
    axes[row, col].set_title(f'Cluster {i+1}')

plt.tight_layout()
plt.show()
```



1.6 1.6

1.6.1 An important part of unsupervised learning is labelling. Manually label the printed images of question (5) and store them in a vector. Then, assign the test data to the 20 clusters and assign the corresponding label to each test instance in the clusters (label propagation). By treating these assignments as predictions compute the accuracy and F1 score, with respect to test set labels, and compare to previous approaches with respect also to training times.

Manually label the printed images of 1.5 and store them in a vector

[32]:
```
cluster_labels = np.array([0, 1, 1, 0, 3, 9, 3, 7, 6, 5, 8, 2, 0, 2, 2, 7, 6,␣
↪9, 7, 4])  # Manually labeled based on printed digits
```

Assign labels to test data based on their cluster assignments

Transform the test data using the same scaler and PCA as training

[33]:
```
X_test_pca = pca.transform(scaler.transform(X_test))
```

Predict clusters for the test data

[34]:
```
test_clusters = kmeans.predict(X_test_pca)
```

Assign labels to test data, based on their cluster assignment

[35]:
```
test_labels_pred = [] # Initialize an empty list to store the predicted labels␣
↪of the test data
for cluster in test_clusters:
    test_labels_pred.append(cluster_labels[cluster]) # For each predicted␣
↪cluster index in the test set, append the previously assigned label to the␣
↪list
```

Convert the Python list of predicted labels into a NumPy array

[36]:
```
test_labels_pred = np.array(test_labels_pred)
```

Evaluate the results

[37]:
```
accuracy_label_propagation = accuracy_score(y_test, test_labels_pred)
f1_macro_label_propagation = f1_score(y_test, test_labels_pred, average='macro')

print(f"Accuracy with Label Propagation: {accuracy_label_propagation:.4f}")
print(f"F1 Score (macro) with Label Propagation: {f1_macro_label_propagation:.
 ↪4f}")
print(f"KMeans Training Time (s): {kmeans_training_time:.4f}")
```

```
Accuracy with Label Propagation: 0.6645
F1 Score (macro) with Label Propagation: 0.6516
KMeans Training Time (s): 0.2565
```

[38]:
```
all_results = {'Algorithm': ['Decision Tree', 'PCA + Decision Tree', 'Gradient␣
 ↪Boosting (PCA)', 'Label Propagation (KMeans + PCA)'],
               'Accuracy': [accuracy, accuracy_pca, accuracy_gbc,␣
 ↪accuracy_label_propagation],
               'F1 Score (macro)': [f1_macro, f1_macro_pca, f1_macro_gbc,␣
 ↪f1_macro_label_propagation],
               'Training Time (s)': [dt_training_time, pca_dt_training_time,␣
 ↪gbc_training_time, kmeans_training_time]}

df_all_results = pd.DataFrame(all_results)
df_all_results['Training Time (s)'] = df_all_results['Training Time (s)'].
 ↪round(1)
df_all_results
```

[38]:

	Algorithm	Accuracy	F1 Score (macro)	Training Time (s)
0	Decision Tree	0.6760	0.663865	0.2
1	PCA + Decision Tree	0.6720	0.668193	2.2
2	Gradient Boosting (PCA)	0.7940	0.790264	23.4
3	Label Propagation (KMeans + PCA)	0.6645	0.651647	0.3

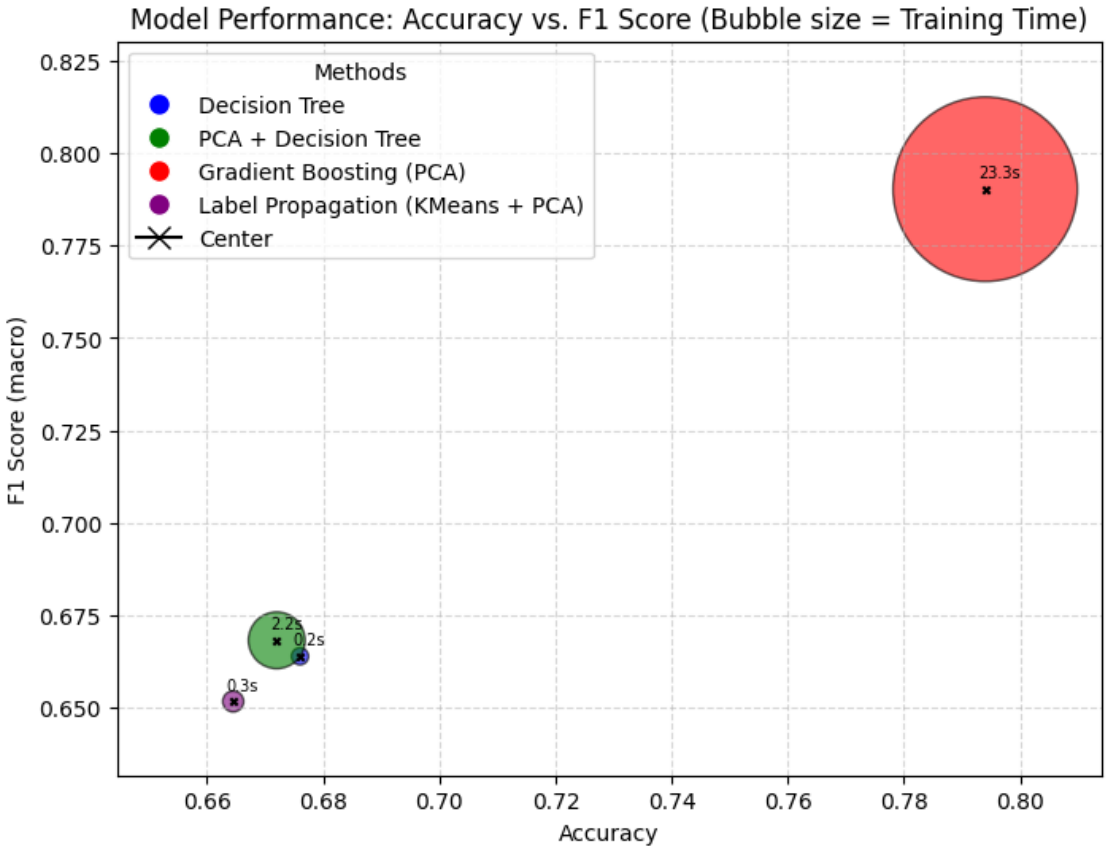
The Decision Tree, even though a fast option, appears to be struggling with complex patterns in the data, as indicated by its accuracy and F1 scores.
The use of PCA does not appear to improve the Decision Tree's performance, suggesting that the dimensionality reduction might have led to the loss of valuable features or that the tree-based model does not benefit significantly from PCA-transformed data.
Gradient Boosting significantly outperforms the Decision Tree models, achieving the best accuracy and F1 score, demonstrating the power of ensemble methods in improving generalization. However, this comes at a cost of a much longer training time.
Finally, event though Label Propagation with KMeans and PCA is computationally efficient, it is the worst performing algorithm, probably because clustering methods do not take into account

label-specific information when forming groups, leading to clusters that may not align well with the actual digit classes. Additionally, the dimensionality reduction from PCA might have removed key features needed for better differentiation

1.6.2 Extra code: Visualization

[39]:
```
# EXTRA CODE #

# Define the results
algorithms = [
    "Decision Tree",
    "PCA + Decision Tree",
    "Gradient Boosting (PCA)",
    "Label Propagation (KMeans + PCA)"
]

accuracy = [0.6760, 0.6720, 0.7940, 0.6645]
f1_score_macro = [0.663865, 0.668193, 0.790264, 0.651647]
training_time = [0.2, 2.2, 23.3, 0.3]

# Define colors for each algorithm
colors = ["blue", "green", "red", "purple"]

# Define sizes for training times
size_factor = 300  # Scaling factor for better visualization
sizes = [t * size_factor for t in training_time]

plt.figure(figsize=(8, 6))
scatter = plt.scatter(accuracy, f1_score_macro, s=sizes, alpha=0.6, c=colors,␣
 ↪edgecolors="black", label=algorithms)

# Add time labels
for i, time in enumerate(training_time):
    plt.annotate(f"{time:.1f}s", (accuracy[i], f1_score_macro[i]), fontsize=7,␣
 ↪xytext=(-3, 5), textcoords="offset points")

# Use X markers for the points, where each point's coordinates are the␣
 ↪corresponding accuracy and F1 score pair.
plt.scatter(accuracy, f1_score_macro, color="black", marker="x", s=10,␣
 ↪label="Center")

# Expand axis limits
plt.xlim(min(accuracy) - 0.02, max(accuracy) + 0.02)
plt.ylim(min(f1_score_macro) - 0.02, max(f1_score_macro) + 0.04)

# Add Labels and title
plt.xlabel("Accuracy")
```

```
plt.ylabel("F1 Score (macro)")
plt.title("Model Performance: Accuracy vs. F1 Score (Bubble size = Training␣
 ↪Time)")

# Add legend
legend_patches = [plt.Line2D([0], [0], marker='o', color='w',␣
 ↪markerfacecolor=colors[i], markersize=10, label=algorithms[i]) for i in␣
 ↪range(len(algorithms))]
legend_patches.append(plt.Line2D([0], [0], marker='x', color='black',␣
 ↪markersize=10, label="Center"))
plt.legend(handles=legend_patches, title="Methods")

# Show plot
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



The visualization confirms the previous results by clearly illustrating the trade-offs between accuracy, F1 score, and training time for each model. Gradient Boosting (PCA) achieves the highest accuracy and F1 score but has a significantly longer training time. The Decision Tree and PCA + Decision Tree exhibit similar performance, with PCA providing no meaningful improvement while

slightly increasing training time. Finally, Label Propagation (KMeans + PCA) is the fastest but delivers the worst performance.